StateTest

ref.

1. Scrutinizing and Improving Impossible Differential Attacks: Applications to CLEFIA, Camellia, LBlock and Simon (Full Version)
2. Improved Differential Meet-in-the-Middle Cryptanalysis
3. The State-Test Technique on Differential Attacks: a 26-Round Attack on CRAFT and Other Applications
4. Improved Differential Meet-In-The-Middle Cryptanalysis on SIMON and Piccolo (Full Version)
5. Probabilistic Extensions: A One-Step Framework for Finding Rectangle Attacks and Beyond

展开 StateTest

使用逻辑

设主密钥长 K 比特, 所有涉及的独立密钥信息为 k_{inv} 比特; st 比特状态 z 压缩了 $|k_{st}|$ 比特密钥, 且 $st < |k_{st}|$. 固定 pair 和其他已猜密钥后, 假设密钥到状态的映射均衡, 检查只通过 z 使用这部分密钥, 且没有额外约束. 每个状态对应 $2^{k_{st}-st}$ 种密钥取值.

优势来源

先猜测状态并过滤, 再**只展开存活状态, 被过滤掉的状态所对应的整组密钥无需逐一处理**.

例子1, 以 ID 为例 (ref.1):

- 准备的 pairs 数为 N. 总涉及密钥 $k_{inv} = 20$ 比特, 将 $|k_{st}| = 8$ 比特密钥压缩为 $st = 4$ 比特状态 z.
- 计入原例的 16 比特过滤条件，并省略相同的单次处理成本，猜测过滤阶段的复杂度由 $(N + 2^{20} \cdot N2^{-16} = 17N)$，降为 $(N + 2^{20-8+4} \cdot N2^{-16} = 2N)$。
- 记 16 比特 $(k_{inv}/k_{st}, z)$ 存活的概率为 P, 则存活的候选密钥数量由 $2^{20}P$ 变为 $2^{16}P$, 但 $2^{20}P$ 包含完整的密钥信息, 而 $2^{16}P$ 有 8 比特密钥被压缩.
- 穷搜时, 需要基于存活的候选密钥进行补全和搜索:
  - 未使用 state test: $2^{20}P \cdot 2^{K-20} = 2^K P$;
  - 使用 state test, 需要展开 z, 即遍历其中 $|k_{st}| - st$ 比特密钥, 如 k_{st1}, 利用关系 $z = func(k_{st1}, k_{st2})$ 每次唯一推导出剩余密钥. 这里, 遍历 4 比特密钥 k_{st1}, 每次唯一推导另外 4 比特 k_{st2}. 此时涉及的密钥信息已经全部得到. 穷搜复杂度变为 $2^{16}P \cdot 2^4 \cdot 2^{K-20} = 2^K P$.

注意: 优势来源在于猜测过滤过程可以通过 *处理压缩成的更短的状态代替处理更长的密钥*, 从而降低复杂度, 且 猜测 压缩的状态 与 被压缩的密钥 对候选密钥的过滤效果相同, 即 P 不变, 所以穷搜的复杂度也不变. (所有 pairs 的相关明文半字节固定为同一值，使密钥与状态的对应关系在这些 pairs 之间保持一致.)

例子 2，以 D-MITM 对 23 轮 CRAFT 的攻击为例（*Improved Differential Meet-In-The-Middle Cryptanalysis*，2024/351，§6.1、附录 E）：

- 主密钥长度为 ($K = 128$) 比特。按上下两侧合并、去重后的计数，显式猜测 ($32 + 32 - 24 = 40$) 比特密钥信息，用 ($st = 28$) 比特状态代替其余 ($|k_{st}| = 88$) 比特密钥的直接枚举。

- 生成与匹配阶段处理这些压缩候选。结合论文的其他攻击步骤，该阶段复杂度约为 (2^{108})，最终留下约 (2^{108}) 条候选记录 $(pair, guessedkey, testedstate)$，同一个 pair 可以对应多条记录。
- 匹配同时*确定原先未猜测的 44 比特密钥 (具体过程略)*，使每条记录具有 $(40 + 44 = 84)$ 比特直接密钥信息，以及 28 比特**状态方程**信息。这 44 比特由匹配关系直接得到，*不需要额外枚举，其恢复成本包含在匹配阶段*。
- 展开 state test 时，结合上述密钥信息求解状态方程，每条记录平均对应约 $(2^{88-44-28} = 2^{16})$ 个完整密钥。作者采用分组、共享求解和列表匹配；在论文给定的一种参数选择下，展开的总成本约为 $(2^{124.58})$ 次小规模运算，均摊到每条记录约为 $(2^{16.58})$。
- 最后逐一验证这些完整密钥，候选检查量约为 $(2^{108} \cdot 2^{16} = 2^{124})$。此时每个候选已经包含全部 128 比特主密钥，*不需要再补猜*其他密钥位。

注意：优势首先来自用**较短的状态表示降低前面的候选生成和处理成本**；**匹配又确定了 44 比特信息，使后续展开只需处理剩余解。**展开成本仍须单独计算，不能仅根据少猜了多少位，就认定整个攻击降低了相同的指数。

成功率/穷搜复杂度 影响

- 以上两种**完整展开**情况, 在不使用密钥计数器, 使用 逐条测试方法
  - 并不会降低攻击的成功率 $1 - e^{-s}$;
  - 穷搜复杂度为猜测过滤后的 $(pair, cand.key, testedstate)$ 数量 $N' \times$ **展开 state test 复杂度** m $\times$ 补全主密钥复杂度 $2^{K-k_{involve}}$, 其中 展开 state test 复杂度 视具体情况而定 (一般会倾向于 test 满足展开时复杂度 $m \leq 2^{k_{st}-st}$ 的状态). 一般不会让穷搜的复杂度变高, 最差情况, 达到与不使用 state test 相同的穷搜复杂度.
- 使用计数器为密钥计数 (阈值筛选方法):
  - 如果 state test 的展开发生在为密钥计数**之前**, 也不影响攻击的成功率和穷搜复杂度, 其对 cand. key 的计票与不使用 state test 相同.
  - 如果 state test 的展开发生在为密钥计数**之后** (**不展开?**), 将丢失 部分密钥计票 (correct/wrong cand. key), 且展开无法补上.

    详细分析 (数据量, 成功率, 穷搜 对比):

    - p: 一个 pair 给正确候选子密钥投票的概率. 在前述假设下, 就是区分器概率.
    - q: 一个 pair 给某个固定错误候选子密钥投票的概率, 由具体检查条件和错误密钥模型决定.
    - 因此, 使用 N 对有效数据时, 正确密钥的期望票数为 $s = Np$ ($N = s/p$), 每个错误密钥的期望票数为 Nq.

    下面仅作数值示例: 假设 p, q 固定, 且在 $s = 3$ 时, $Nq = 0.1$. 当 s 增至 6.3 时, 数据量增至原来的 2.1 倍, 相应地, Nq 增至 0.21.

    设主密钥长度为 K 比特. 按前述具体子密钥计数模型, 当保留的错误候选占主导时 (正确密钥只有一个), 期望穷搜复杂度约为 $\mathbb{E}[T_{\text{exhaustive}}] \approx 2^K \beta$.

    采用泊松近似, 结果如下:

数据参数 s	阈值 θ	正确密钥保留概率(成功率)	单个错误密钥保留概率 β	期望穷搜复杂度
3	1	95.0%	9.52%	$2^{-3.39} \cdot 2^K$
3	3	57.7%	0.0155%	$2^{-12.66} \cdot 2^K$
6.3	3	95.0%	0.132%	$2^{-9.57} \cdot 2^K$

这里有两种不同的比较:

- **第一行与第二行: 数据相同.** 提高阈值大幅减少错误候选, 但正确密钥也更容易被删掉, 因此成功率下降, 穷搜复杂度降低.
- **第一行与第三行: 成功率约相同.** 将数据增至约 2.1 倍, 补偿提高阈值造成的成功率下降. 此时错误密钥也会获得更多票, 所以 β 比第二行增大, 穷搜复杂度增加, 但仍明显低于第一行, 数据复杂度也同步升高.

另外, $\theta = 1$ 本身不降低相对于逐条测试的成功率; 它还可以通过**同一候选只补全验证一次**, 减少重复工作.

详细分析不展开情况

ref. Probabilistic Extensions: A One-Step Framework for Finding Rectangle Attacks and Beyond

记 $k_{involve}$ 可被分为 k_1 猜测, k_2 用 tested state 压缩为 st 比特状态 z. **只为 k_1 建立密钥计数器.** 记 $F_i(k_1, z) = 1$ 表示第 i 个 pair 在该猜测下满足区分器差分, 记 cand. key 保留条件为 $C(k_1) \geq \theta$:

$$C(k_1) = \sum_{i=1}^{N} \mathbf{1}[\exists z:\ F_i(k_1, z) = 1].$$

对正确 pair, 真实的 tested state 一定在枚举范围内, 所以它仍然给正确的 k_1 投票.

变化主要在于: 错误 pair 也可能借助某个状态值通过检查, 使噪声 (给错误密钥的投票) 增加.

所以, 记未使用 state test 的错误密钥计票为 e, 使用 state test 的错误密钥计票为 e'. 则有 $e' = 2^{st}e$, **即错误密钥的计数增加 2st.**

用一个假想的单次检查作为基准: 对每个 k_1, 每个 pair 只检查一个固定状态, 记固定 (k_1, z) 后的噪声通过概率为 filter, 则总噪声投票量约为 $2^{k_1}N \cdot filter$. (**额外**) 枚举 2st 个 tested states 后, 各状态通过检查的结果都汇入同一个 k_1 计数器. 在均匀噪声模型下, 若不同状态同时通过的重叠影响可忽略, 总噪声投票量约增至 $2^{k_1+st}N \cdot \mathrm{filter}$. 由于计数器数量仍为 2^{k_1}, 每个计数器的平均噪声约增加 2st 倍.

项目	不使用该技术	使用该技术后的变化
计数器数量	$2^{k_1+k_2}$	变为 2^{k_1}, 减少 2^{k_2} 倍
总噪声投票量	$2^{k_1+k_2}Nq$	约为 $2^{k_1+st}Nq$, 减少 2^{k_2-st} 倍
每个计数器的平均噪声	$e = Nq$	约为 $e' = 2^{st}Nq = 2^{st}e$, 增加 2st 倍

这时:

- 正确密钥计数期望是 $s = Np$ (p 是区分器概率).
- 错误密钥计数期望是 $e = Nq \to N/2^n$.
- 使用 state test 之后, 错误密钥计数期望 $e' = 2^{st}e$.

考察正确的部分子密钥 k_1 对应的计数器. 设其余密钥为 k_2, 正确取值设为 k_2^*, tested state 为 $z = f(k_2, d)$, 其中 d 是由 pair 猜测过滤后决定的已知数据. 每条通过检查的记录 $(pair, k_1, z)$, 展开后对应满足状态方程的 k_2 取值集合. 假设得到以下两条记录:

通过检查的记录	展开后相容的 k_2 集合	给正确 k_1 的票
正确 pair 的真实状态记录	包含 k_2^*	1 张真票
错误 pair 的噪声记录	不包含 k_2^*	1 张噪声票

第二条记录虽然不对应真实的 k_2^*, 但它通过了当前检查. 由于计数器只以 k_1 为索引, 尚未区分 k_2, 该记录仍给正确 k_1 加一票. 因此, 正确 k_1 的计数为 **1 张真票 + 1 张噪声票 = 2 张票**. 若真票和噪声票的期望分别为 s 和 e', 总计数的期望就是 $s + e'$.

> 固定正确的部分子密钥 k_1 并设置计数器, 设其余两位密钥为 $k_2 = (u, v)$, 正确取值为 00. tested state 为 $z = u \oplus v \oplus d$, 其中 d 是随 pair 变化的已知比特. 假设筛选后得到以下两条记录:
>
> | pair | 已知 d | 通过检查的状态 z | 展开后相容的 k_2 | 给正确 k_1 的票 |
> |---|---|---|---|---|
> | 正确 pair | 0 | 0 | $\{00, 11\}$ | 1 张真票 |
> | 错误 pair | 1 | 0 | $\{01, 10\}$ | 1 张噪声票 |
>
> 第二条记录中的状态 $z = 0$ 不对应正确的 $k_2 = 00$, 但它通过了当前检查. 由于计数器只以 k_1 为索引, 尚未区分 k_2, 这条记录仍给正确 k_1 加一票. 因此, 正确 k_1 的计数为 **1 张真票 + 1 张噪声票 = 2 张票**. 一般地, 若真票和噪声票的期望分别为 s 和 e', 总计数的期望就是 $s + e'$.

所以正确密钥和错误密钥的得票数为:

$$C(k_1^{\text{true}}) \approx \text{Pois}(s + e'), \qquad C(k_1^{\text{wrong}}) \approx \text{Pois}(e').$$

记 $Q_\theta(\lambda) = 1 - e^{-\lambda} \sum_{j=0}^{\theta-1} \lambda^j / j!$ (泊松分布展开)

所以有

$$\boxed{P_{\text{success}} \approx Q_\theta(s + e'), \qquad \beta_{st} \approx Q_\theta(e'), \qquad e' = 2^{m_e} N q.}$$

正确 k_1 过阈值后, 后续展开和补全一定覆盖真密钥. 若忽略正确计数器上的噪声 (可忽略), 成功率可保守地按 $Q_\theta(s)$ 估计, 一般 $e' = s2^{m_e} \ll s$. 即**成功率不变**. 也即, 对正确密钥的计数保持, 正确密钥可以通用通过阈值.

但 β_{st} 增大, 更多错误密钥通过阈值, 需要补全与穷搜 (穷搜复杂度: $2^K \beta_{st}$). 即噪声增加.

$$\boxed{T_{\text{exhaustive}} \approx 2^{|k_1|} \beta_{\text{ST}} \cdot 2^{K-|k_1|} = 2^K \beta_{\text{ST}}.}$$

对最后穷搜的影响: $T_exhaustive = 2^{K-h}$

- **固定 (h)**: 穷搜预算仍是 2^{K-h}, 噪声增加可能体现为**成功率下降**.
- **保持目标成功率**: 可能需要保留更多候选, 即减小 h. 若调整为 h_{st}, 穷搜量增加的倍数为

  $$\frac{2^{K-h_{st}}}{2^{K-h}} = 2^{h-h_{st}}.$$

  当 h_{st} 仍然可以保证穷搜不为主项, state test 不展开（穷搜后展开）对复杂度无影响.